Les arbres

Structure de données très importante, utilisée à de multiples reprises en informatique. Les traitements sur les arbres font appel à la récursivité.

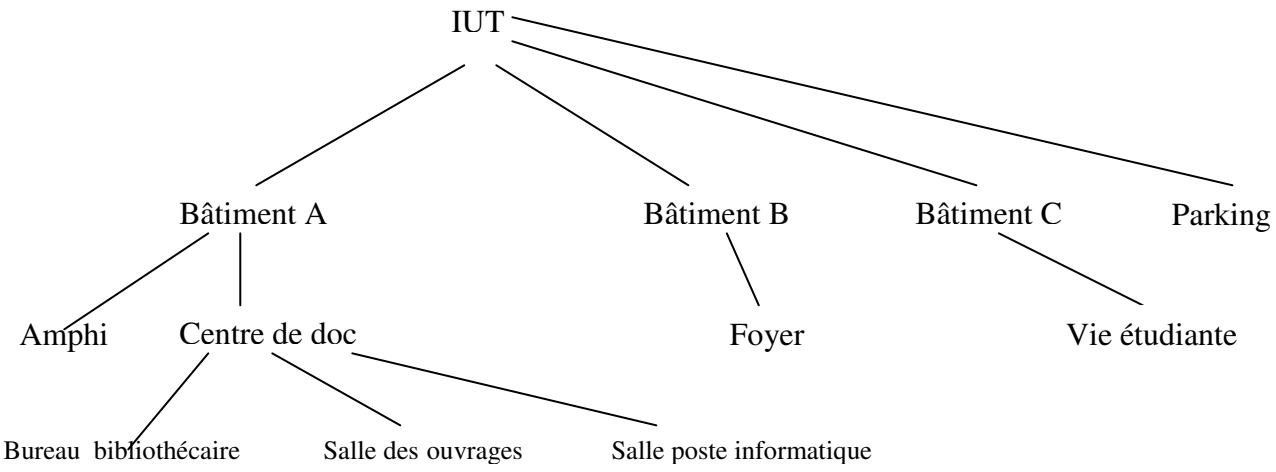
I) Définitions - Exemples

1.1) Définitions

Un **arbre** est une structure de données composée d'un ensemble de **nœuds**. Chaque nœud contient :
- une information particulière parmi celles que l'arbre sert à représenter,
- et des liens vers d'autres nœuds (que l'on nomme des sous-arbres).

Exemple d'un arbre n-aire

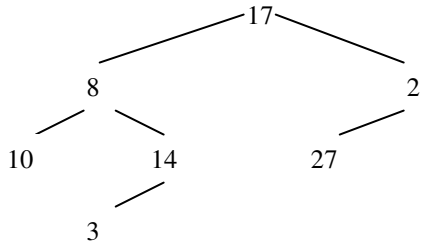
On souhaite développer une application qui serait installée sur des bornes interactives et qui permettrait aux usagers de situer rapidement un lieu dans les locaux de l'IUT. Il serait pratique de représenter les lieux sous forme arborescente. Le schéma ci-dessous est un arbre qui modélise une partie seulement des locaux de l'IUT.



Vocabulaire

Notion	Définition	Exemples
feuille	Les **feuilles** sont les nœuds qui ne pointent sur aucun autre nœud.	Amphi, bureau bibliothécaire, salle des ouvrages, salle poste informatique, Parking ...
racine	La **racine** de l'arbre est le nœud qui n'est référencé par aucun autre nœud. La racine est unique.	IUT
niveau d'un nœud	Le niveau de la racine est 1. Les autres nœuds ont un niveau qui est augmenté de un par rapport au nœud dont ils descendent.	Le niveau de Parking est 2, celui de salle poste informatique est 4.
hauteur	La **hauteur de l'arbre** est le niveau le plus élevé qui existe dans l'arbre.	Dans l'exemple, la hauteur de l'arbre est 4.
degré d'un nœud	Le **degré d'un nœud** est le nombre de successeurs de ce nœud	Dans l'exemple, il y a des nœuds de degré 0, 1, 3 et 4
degré d'un arbre	Si le n est le degré maximum des nœuds de l'arbre, l'arbre est dit **n-aire**. On dit aussi que l'arbre est de **degré n**.	L'arbre de l'exemple est de degré 4.
arbre ordonné	Si l'ordre des sous-arbres est significatif, on dit que l'arbre est ordonné.	L'arbre de l'exemple n'est pas ordonné. Exemple d'arbre ordonné : un arbre généalogique
arbre binaire	Un **arbre binaire** est un type d'arbre ordonné tel que chaque nœud a au plus 2 fils et quand il y en a un seul, on précise s'il s'agit du fils gauche ou droit.	L'arbre de l'exemple n'est pas binaire.

Exemple d'arbre binaire : on constate que chaque nœud a 0, 1 ou 2 descendants (ou successeurs).



1.2) Exemples d'utilisation des arbres

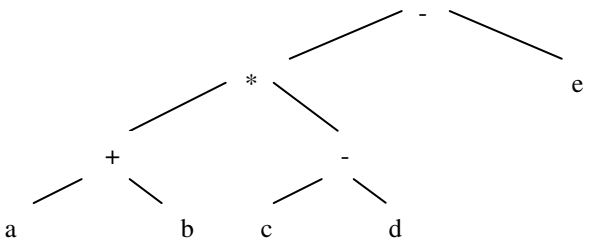
Les arbres sont des structures de données qui sont naturellement utilisées pour des situations de la vie courante indépendamment du domaine de l'informatique : arbres généalogiques, classifications en biologie, structure d'un bâtiment …

En informatique, on les utilisera bien sûr pour modéliser ces situations. Mais on les utilise aussi pour modéliser des informations qui naturellement ne sont pas arborescentes (ou pas tout à fait arborescentes) mais qui une fois modélisées sous forme d'arbre rendent les traitements plus faciles à écrire ou permettent de les optimiser.

a) Expression arithmétique : arbre binaire ordonné

Une expression arithmétique formée d'opérateurs binaires peut être schématisée sous la forme d'un arbre binaire.

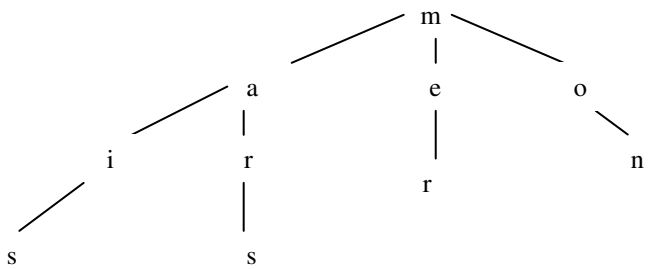
Par exemple : ((a + b) * (c - d) - e)



b) Représentation de mots

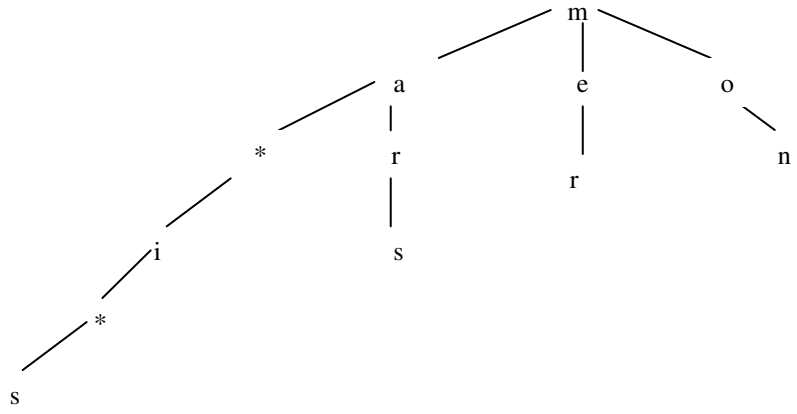
On souhaite représenter une liste de mots en occupant un minimum d'espace en mémoire, et en facilitant les recherches permettant de savoir si un mot est présent ou pas dans la liste.

Exemple : les mots de la liste sont : « mais », « mars », « mer », « mon ». On définit un arbre dont les nœuds contiennent un unique caractère, donc une lettre du mot présent dans la liste. On optimise en occupation car par exemple, la lettre 'm' située au début de chaque mot est stockée une seule fois, la lettre 'a', 2ème lettre de 2 mots, est elle aussi stockée une seule fois.



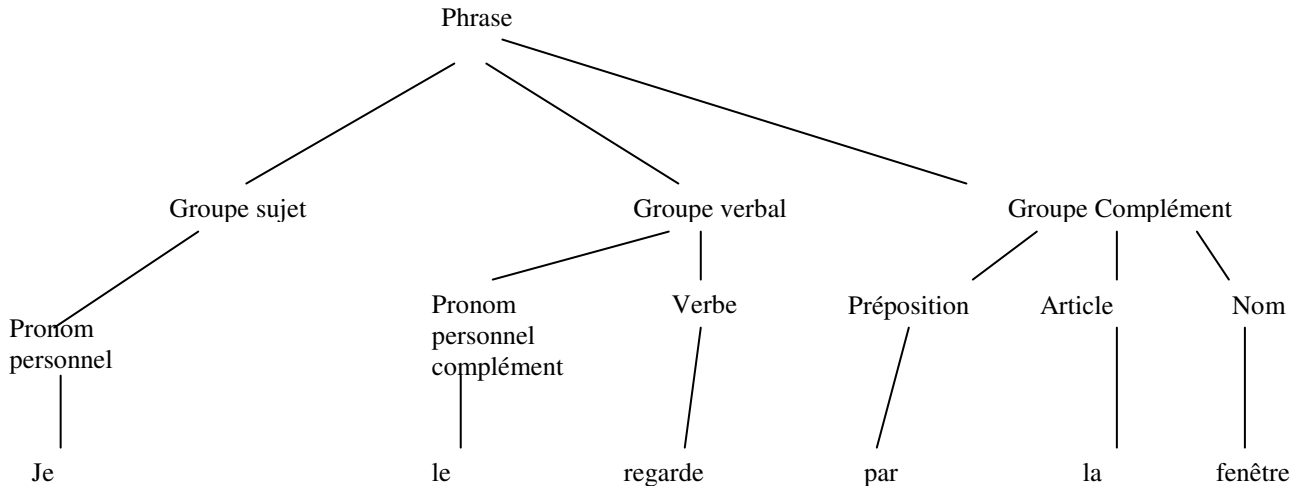
Si on considère que les mots « mai» et « ma » font aussi partie de la liste, il faut éventuellement ajouter une information sur chaque nœud (qui n'est pas une feuille) selon que le nœud est une terminaison

de mot ou pas. Ou bien ajouter des nœuds à l'arbre et qui contiendront un caractère spécial indiquant la fin d'un mot, comme le caractère '*' par exemple. On obtient alors l'arbre suivant :



c) Structure d'une phrase : arbre n-aire ordonné

Structure d'une phrase : "Je le regarde par la fenêtre"



Ce type de structure peut être utilisé par les programmes permettant d'effectuer des traductions automatiques dans une autre langue, prononcer une phrase (l'intonation doit changer selon la structure de la phrase), comprendre le sens de la phrase et agir en conséquence (par exemple pour commander un robot).

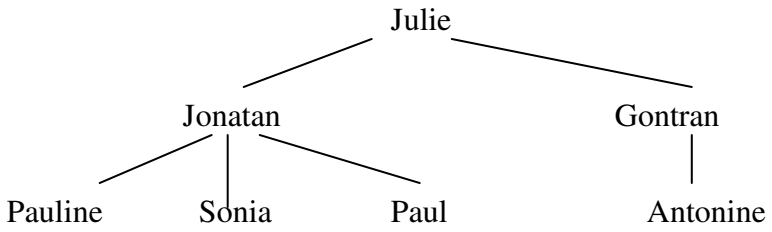
d) Arbre généalogique : arbre n-aire

Il permet de montrer la descendance d'une personne.

Exemple : Julie a 2 enfants : Jonatan et Gontran. Jonatan a 3 enfants : Pauline, Sonia et Paul. Gontran a une fille : Antonine.

Le degré de l'arbre est 3, l'arbre est dit 3-aire, ou n-aire d'une manière générale. Le degré de chaque nœud est variable puisqu'il dépend du nombre d'enfants de ce nœud. Julie est la racine de l'arbre.

L'arbre est ordonné si les nœuds fils sont placés dans l'ordre de naissance.



e) Autres exemples :

- un tournoi de tennis pour montrer qui doit jouer contre qui
- structure en chapitre et sous-chapitres, sections, sous-sections ... d'un document
- la notion de répertoire au niveau d'un système d'exploitation. Les fichiers sont les feuilles. Les dossiers (ou répertoires) sont les noeuds
- l'interface graphique d'un logiciel : fenêtres, sous-fenêtres (ou panneaux en Java par exemple), composants ...
- hiérarchie des classes en Java
- les composants d'un objet matériel : exemple ci-dessus l'IUT. Autre exemple : une voiture (moteur, carrosserie, sièges ....)
- les classifications en biologie :
  - ✓ les espèces animales (vertébrés : poissons, reptiles, oiseaux, mammifères ...et invertébrés : arthropodes : crustacés, insectes, arachnides ..., vers, mollusques ...),
  - ✓ les végétaux

1.3) Brève présentation de la représentation en mémoire des arbres n-aires

Il faut donc mémoriser les valeurs des nœuds et les relations de dépendance père-fils.

Exemple arbre l'arbre généalogique. Nous allons envisager 3 représentations possibles.

a) Mémorisation par liste de fils

L'allocation est en partie contiguë (éléments placés les uns à la suite des autres en mémoire) et en partie dynamique (les éléments sont chaînés les uns aux autres).

Dans une table (tableau) on stocke tous les éléments de l'arbre, ils sont implicitement numérotés par leur indice dans le tableau. L'ordre n'est pas significatif. Les relations père-fils sont stockées de manière dynamique : à chaque élément de la table est associé une liste de descendants dans laquelle seuls les numéros sont stockés. Cette liste est vide pour les nœuds feuilles.

Avantages : Facilité pour accéder aux fils d'un nœud

Inconvénients : difficile de donner un maximum pour la taille de la table (mais bien si on connaît ce maximum). Les insertions et suppressions ne sont pas faciles à gérer, pour la partie contiguë.

b) Allocation dynamique

Les éléments sont alloués au cours de la construction de l'arbre et reliés entre eux. Dans chaque nœud, on va trouver des pointeurs pour représenter la relation père-fils (vous implémenterez cette représentation en TP).

Si le degré de chaque nœud est constant, cette représentation est bien adaptée.

Une alternative consisterait, en Java par exemple, à associer à chaque nœud une liste de descendants.

c) Allocation contiguë (tableau ou fichier)

Si l'on souhaite stocker entièrement l'arbre dans un espace contigu, cela suppose de réserver l'espace mémoire nécessaire en précisant un maximum pour toutes les entités : nombre de nœuds, et nombre maximum de descendants pour chaque nœud.

0	Julie
1	Jonatan
2	Pauline
3	Sonia
4	Paul
5	Gontran
6	Antoine

1	5	/
2	3	4
/	/	/
/	/	/
/	/	/
6	/	/
/	/	/

Inconvénients : si le nombre de descendants est très variable, il y a une perte de place.

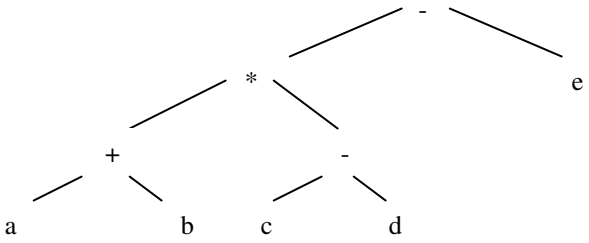
II) Les arbres binaires

2.1) Définition

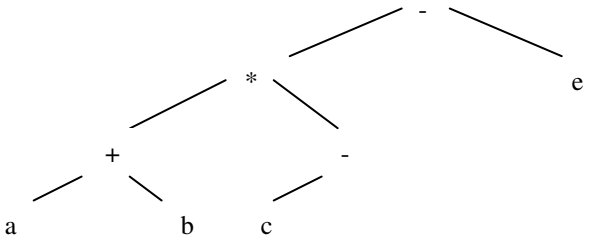
Un arbre binaire est un arbre ordonné tel que chaque nœud a **au plus deux fils** et quand il en a un seul celui-ci est identifié comme étant le **fils gauche** ou le **fils droit**.

Exemple d'arbre binaire. Chaque nœud a :

- soit aucun fils, dans ce cas le nœud est une feuille
- soit 2 fils. Par exemple : le nœud qui contient la valeur '+' a pour fils gauche 'a' et pour fils droit 'b'. Le nœud qui contient la valeur '*' a pour fils gauche '+' et pour fils droit '-'.



L'arbre ci-dessous est aussi un arbre binaire. Chaque nœud a de 0 à 3 fils.

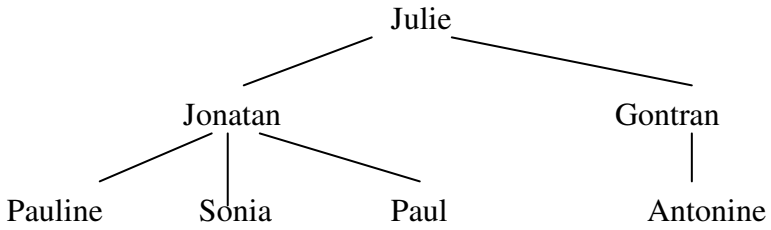


2.2) Transformation d'un arbre n-aire en un arbre binaire

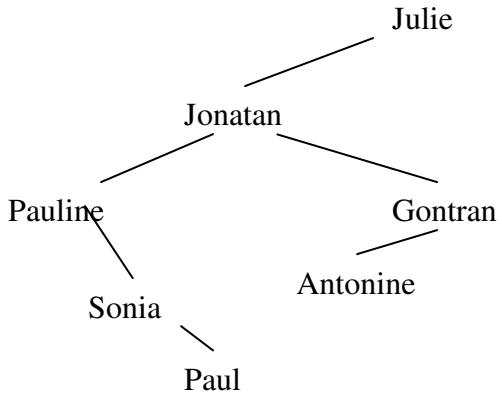
Tout arbre n-arbre peut être transformé en un arbre binaire en appliquant un algorithme bien précis. A partir de l'arbre binaire obtenu, il est possible de reconstituer l'arbre n-aire d'origine. Comme suggéré précédemment, les arbre binaires sont plus faciles à implémenter que les arbres n-aires. Cette transformation facilite la représentation des arbres n-aires : on les transforme d'abord en arbre binaire pour avoir l'assurance qu'un nœud aura au plus 2 descendants.

Cette transformation repose sur l'idée suivante : dans chaque nœud de l'arbre binaire, le fils gauche correspond au premier fils de ce nœud dans l'arbre n-aire, et le fils droit correspond au frère immédiat de celui-ci.

Exemple de l'arbre généalogique, cet arbre est n-aire.



Transformé en arbre binaire, en appliquant la technique expliquée, on obtient :



2.3) Représentation en mémoire d'un arbre binaire

a) Représentation dynamique : création des nœuds au fur et à mesure. Voir schéma ci-dessus

b) Représentation contigüe

Par un tableau en mémoire centrale, ou dans un fichier, et éventuellement un fichier en accès direct.

	Nom	Gauche	Droite
0	Julie	1	/
1	Jonatan	2	5
2	Pauline	/	3
3	Sonia	/	4
4	Paul	/	/
5	Gontran	6	/
6	Antonine	/	/

Exemple expression arithmétique : ((a + b) * (c - d) - e)

	Nom	Gauche	Droite
0	-	1	8
1	*	2	5
2	+	3	4
3	a	/	/
4	b	/	/
5	-	6	/
6	c	/	/
7	d	/	/
8	e	/	/

2.4) Important - Parcours d'un arbre binaire

Un algorithme de parcours d'arbre est une méthode permettant **d'accéder successivement à chacun des nœuds** de l'arbre. On peut au fur et à mesure du parcours effectuer un traitement précis sur chacun des nœuds : test, comptage, vérifier une propriété ...

Exemple sur l'arbre généalogique : rechercher les enfants de Gontran, compter la descendance de Julie, compter le nombre de garçons, ajouter un fils à Paul ...

On distingue 2 catégories de parcours :

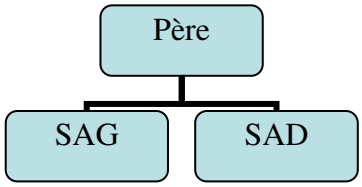
- ✓ les parcours en profondeur : on explore branche par branche
- ✓ les parcours en largeur : on explore niveau par niveau

La récursivité facile l'écriture des traitements sur les arbres, car cette structure de données se définit récursivement.

a) Les parcours en profondeur d'un arbre

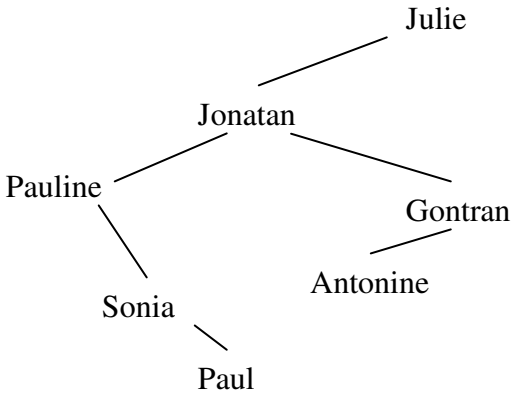
Nous envisagerons ici les parcours gauche-droite. Mais bien sûr, il existe des parcours symétriques droite-gauche. Il existe 3 types de parcours en profondeur dans le sens gauche-droite.

Nous utiliserons les abréviations suivantes : P = Père, SAG = sous-arbre gauche, et SAD = sous-arbre droit.



	Gauche-droite	*Droite-gauche*
Préfixé	**P - SAG - SAD**	*P - SAD - SAG*
Infixé	**SAG - P - SAD**	*SAD - P - SAG*
Postfixé	**SAG - SAD - P**	*SAD - SAG - P*

Exemple avec l'arbre généalogique binaire



Parcours préfixé : on traite le nœud lors de sa première visite, puis on traite le sous-arbre gauche (en appliquant le même algorithme), puis le sous-arbre droit (toujours avec le même algorithme).
On commence par le nœud racine « Julie », puis on descend à gauche pour traiter le descendant à gauche s'il existe, ici « Jonatan ». On continue à descendre à gauche où on trouve « Pauline ». Ce nœud n'a pas de descendant à gauche. Donc on va parcourir sa descendance à droite …

Julie Jonatan Pauline Sonia Paul Gontran Antonine

Parcours infixé : le nœud est traité lors de sa deuxième visite, après avoir traité le sous-arbre gauche, mais avant de traiter le sous-arbre droit.

Pauline Sonia Paul Jonatan Antonine Gontran Julie

Parcours postfixé : le nœud est traité lors de sa troisième visite, après avoir traité le sous-arbre gauche, et le sous-arbre droit.

Paul Sonia Pauline Antonine Gontran Jonatan Julie

Exemple pour l'expression arithmétique

Parcours préfixé	-	*	+	a	b	-	c	d	e
Parcours infixé	a	+	b	*	c	-	d	-	e
Parcours postfixé	a	b	+	c	d	-	*	e	-

b) Les parcours en largeur d'un arbre

Une autre méthode de parcours des arbres consiste à les visiter étage par étage. Cette technique est moins fréquente que la précédente. Ceci est dû au fait que la précédente peut s'appuyer sur la récursivité. Par contre, le parcours en largeur a l'avantage d'être très utile pour dessiner un arbre.

Exemple avec l'arbre généalogique : Julie Jonatan Pauline Gontran Sonia Antonine Paul

Pour l'expression mathématique : - * + e - a b c d

Principe : Ce parcours nécessite l'utilisation d'une file d'attente contenant initialement la racine. On extrait l'élément en tête de la file, et on le remplace par ses successeurs de gauche à droite jusqu'à ce que la file soit vide.

III) Les arbres binaires ordonnés

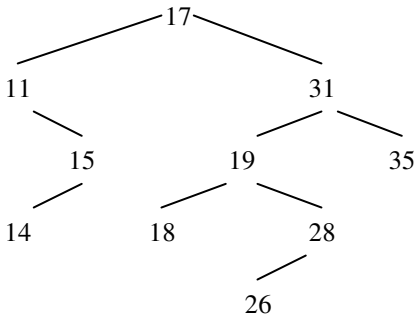
Les arbres binaires ordonnés sont des arbres binaires tels que pour chaque nœud, les clés (identifiants) des éléments du sous-arbre gauche sont inférieures à celle de la racine, et celles du sous-arbre droit sont supérieures à celle de la racine. On nomme aussi ces arbres **: des arbres binaires de recherche.**

Dans les exemples suivants, nous modéliserons des arbres contenant des entiers. Il faut imaginer que cette valeur du nœud est en fait la clé d'une information plus complexe.

Par exemple, l'arbre suivant (figure 1) est un arbre binaire ordonné. Pour s'en assurer, il faut vérifier les 2 points suivants, pour chacun des nœuds :

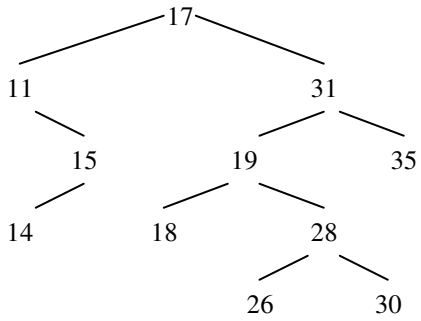
- le fils gauche, s'il existe, a une valeur plus petite que le nœud courant
  11 < 17 14 < 15 19 < 31 18<19 16<28

- le fils droit, s'il existe, a une valeur plus grande que le nœud courant
  15>11 31>17 28>19 35>31



Question : Où faut-il ajouter 30 à l'arbre ? 30 a une place unique qu'il faut trouver en suivant le raisonnement ci-dessous :

- plus grand que 17, donc à sa droite
- plus petit que 31, donc à sa gauche
- plus grand que 19, donc à sa droite
- plus grand que 28, donc à sa droite. Comme 28 n'a pas de descendant à droite, 30 devient le descendant à droite de 28.



Avantage

Pour retrouver un élément dans un arbre binaire de recherche, il n'est pas nécessaire de parcourir tout l'arbre. A chaque nœud, on décide de continuer la recherche soit dans le sous-arbre gauche, soit dans le sous-arbre droit.

Par exemple, supposons que l'on cherche la valeur 15 dans l'arbre ci-dessous. On part de la racine égale à 17. 15 étant plus petit que 17, si 15 est présent il le sera nécessairement à gauche de 17. On descend donc dans le sous-arbre gauche dont la racine est 11. 15 est plus grand que 11, il faut donc descendre dans le sous-arbre droit de 11. On trouve là la valeur 15. 15 est donc présent. Pour aboutir à cette conclusion, on a parcouru les valeurs 17, 11 et 15. Donc seulement 3 valeurs.

Supposons maintenant que l'on cherche la valeur 20. 20 est supérieur à la racine 17. On descend donc à droite de la racine. 31 est plus grand que 20. Donc il faut continuer la recherche à gauche de 31. La racine du sous-arbre gauche est 19. 20 étant plus grand, on le cherche à droite. La racine du sous-arbre droit est 28. 20 est plus petit, on doit donc le rechercher à gauche. La racine du sous-arbre gauche est 26 qui est plus grand que 20. Il faudrait donc chercher 20 à gauche de 26. Or il n'y a pas de fils à gauche de 26. On en déduit que 20 n'est pas présent dans l'arbre. Pour aboutir à cette conclusion, on a parcouru le chemin :

17 31 19 28 26.

Remarques

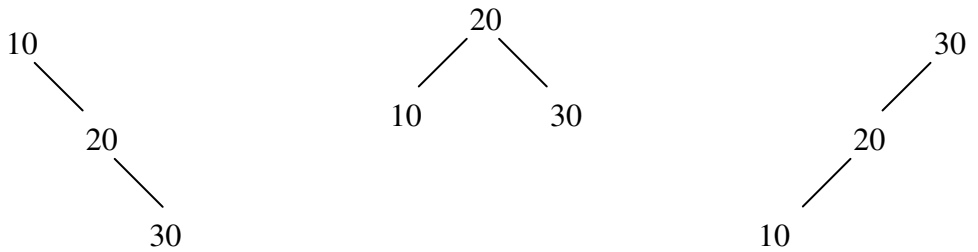
Le parcours infixé gauche-droite de l'arbre donne la liste des éléments dans l'ordre croissant.

La recherche d'un élément connaissant sa clé est grandement facilitée. Il n'est pas nécessaire de parcourir tout l'arbre pour retrouver un élément. A chaque nœud, on peut décider du chemin à suivre, soit dans le sous-arbre gauche, soit dans le sous-arbre droit.

L'insertion se fait toujours au niveau d'une feuille.

La structure de l'arbre dépend de l'ordre dans lequel les éléments sont ajoutés. Par exemple, si l'arbre contient au final 10, 20 et 30. On a pu ajouter dans l'ordre :

10, 20 puis 30 20, 10 puis 30 30, 20 puis 10



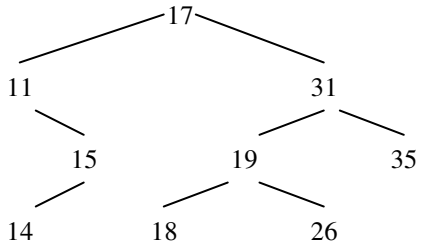
Si les valeurs sont insérées dans l'ordre croissant, on aboutit à un arbre dit "dégénéré", car il s'agit en fait d'une liste.

Suppression d'une valeur de l'arbre

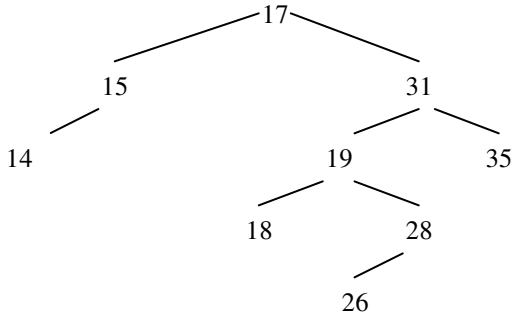
La suppression d'un nœud peut demander une réorganisation de l'arbre. Plusieurs cas sont à envisager au nombre de 4.

Le cas le plus simple est la suppression d'une feuille. Aucune réorganisation n'est exigée.

Si le sous-arbre droit du nœud à supprimer est vide, comme par exemple lors de la suppression de 28 sur la figure 1, il suffit de placer la racine du sous-arbre gauche à la place du nœud à supprimer.

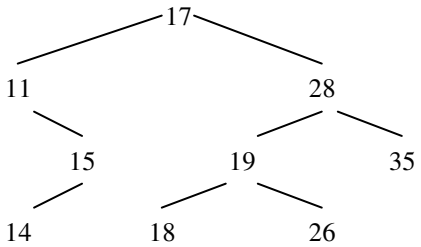


Si le sous-arbre gauche du nœud à supprimer est vide, comme par exemple lors de la suppression de 11, il suffit de placer la racine du sous-arbre droit à la place du nœud à supprimer.



Sinon, dans le cas général aucun des sous-arbres du nœud à supprimer n'est vide. La valeur à supprimer peut être remplacée par la valeur la plus grande du sous-arbre gauche (de manière symétrique, on pourrait décider aussi de la remplacer par la valeur la plus petite du sous-arbre droit).

Supposons que l'on souhaite supprimer 31. La valeur la plus grande du sous-arbre gauche est 28. Il faut donc supprimer 28 de sa place initiale, et le mettre à la place de 31.

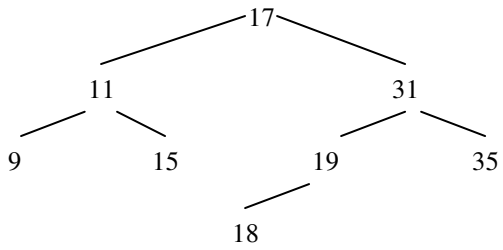


IV) Les arbres binaires ordonnés équilibrés

4.1) Définitions

Un arbre binaire ordonné est dit **parfaitement équilibré** si en tout nœud de l'arbre, le nombre de nœuds du sous-arbre gauche et le nombre de nœuds du sous-arbre droit diffèrent d'au plus 1.

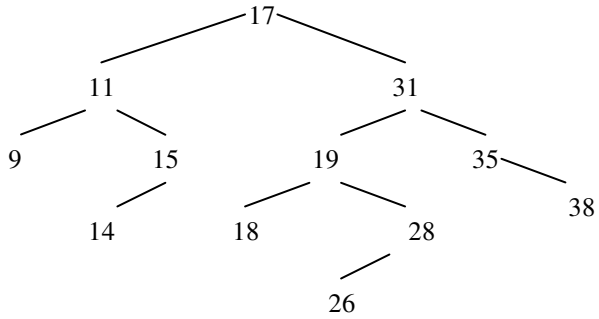
Exemple :



Ce critère est difficile à maintenir car il nécessite des réorganisations importantes de l'arbre lors des ajouts. On utilise plutôt un critère amoindri mais satisfaisant.

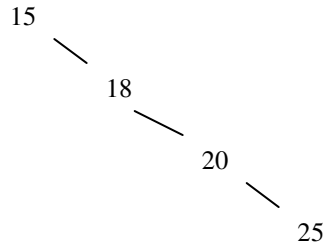
Un arbre binaire ordonné est dit **équilibré** si en tout nœud la hauteur du sous-arbre gauche et la hauteur du sous-arbre droit diffèrent d'au plus 1.

Les arbres équilibrés sont dits aussi balancés ou de type AVL (les initiales des informaticiens qui ont travaillé les premiers sur cette notion, Adelson, Velskii et Landis) .



Pourquoi s'intéresser à la notion d'arbre équilibré ?

Les ajouts et suppressions dans un arbre binaire peuvent le déséquilibrer. On obtient alors un arbre dégénéré dans lequel la recherche d'un élément est dégradée en terme d'efficacité. Dans le pire des cas, elle pourrait devenir équivalente à une recherche dans une liste, comme par exemple dans un arbre tel que celui-ci :



Il existe des méthodes permettant de réorganiser un arbre binaire qui serait devenu déséquilibré.

4.2) Méthode pour réorganiser un arbre binaire déséquilibré

Une réorganisation doit avoir lieu si, à la suite d'une insertion, la valeur absolue de la différence de hauteurs entre la plus longue branche du SAD et celle du SAG d'un nœud devient supérieure strictement à 1.

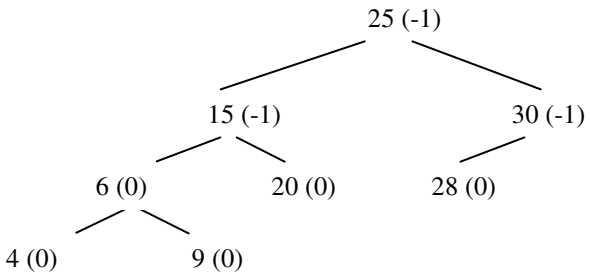
Il y a 4 cas de réorganisations de l'arbre à envisager :

- ✓ le déséquilibre vient du SAG du SAG, noté GG
- ✓ il vient du SAD du SAD, noté DD. Ce cas est symétrique au précédent.
- ✓ il vient du SAD du SAG, noté GD
- ✓ il vient du SAG du SAD, noté DG. Ce cas est symétrique au précédent.

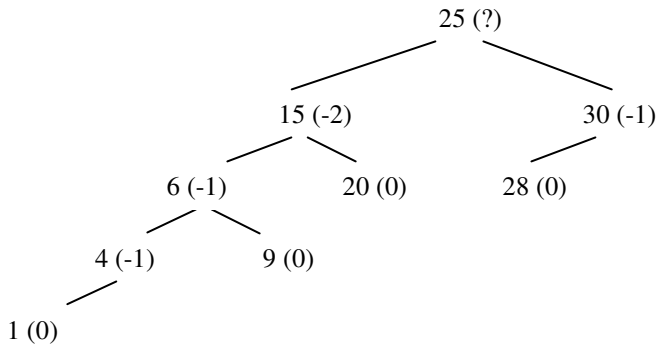
Les méthodes pour rééquilibrer nécessitent de calculer le facteur d'équilibre de chaque nœud. Ce facteur est égal à la hauteur du sous-arbre droit moins celle du sous-arbre gauche. Dans les exemples suivants, ce facteur est noté entre parenthèses.

a) Réorganisation de type GG (le déséquilibre vient du SAG du SAG)

Supposons par exemple que l'arbre initial soit le suivant (il est équilibré)

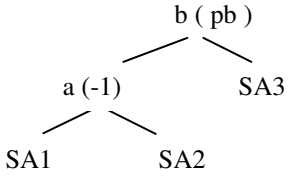


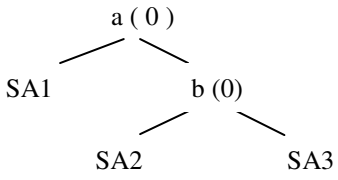
On souhaite insérer 1 dans cet arbre. On obtiendrait :



Le facteur d'équilibre du nœud 15 devient égal à -2. Il faut donc réorganiser l'arbre à partir de ce nœud.

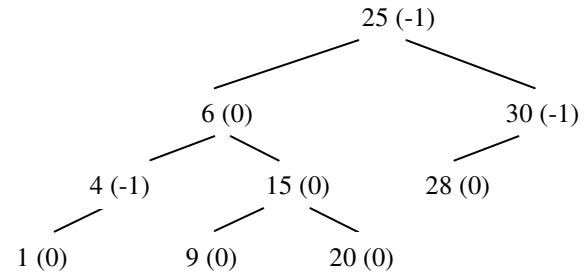
D'une manière plus générale, il faut réorganiser l'arbre de la manière décrit ci-dessous. Le sous-arbre du schéma de gauche doit devenir le sous-arbre du schéma de droite.





Cette transformation correspond à une **rotation droite.**

Sur l'exemple, la réorganisation donnera :



La réorganisation s'effectue au retour de l'appel récursif qui permet d'insérer 1, lors de la remontée vers la racine de l'arbre. En effet, au retour on calcule le nouveau facteur d'équilibre du nœud courant. Sur le nœud 15, ce calcul donne -2 ce qui impose une réorganisation du nœud 15 et de son fils 6. 6 devient racine d'un sous-arbre à la place de 15 qui glisse à droite de 6. Le sous-arbre 9 est rattaché comme SAG de 15. Les autres nœuds de l'arbre ne sont pas concernés par cette réorganisation.

b) Réorganisation de type DD (le déséquilibre vient du SAD du SAD)

Le principe est identique à la réorganisation GG précédente, à la symétrie près.

Le principe général est le suivant : le sous-arbre du schéma de gauche doit devenir le sous-arbre du schéma de droite.

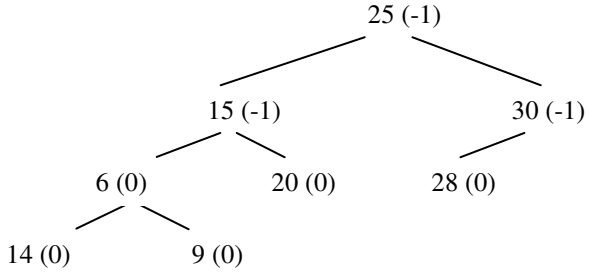


Cette transformation correspond à une **rotation gauche**.

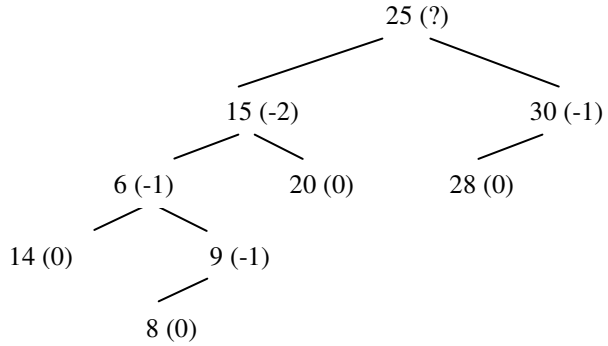
c) Réorganisation de type GD

Le déséquilibre vient de la droite du sous-arbre gauche.

Exemple. Supposons que l'arbre initial soit le suivant (il est équilibré)

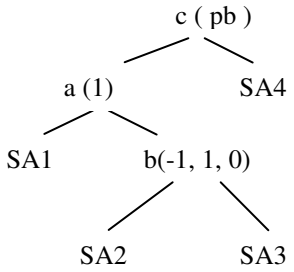


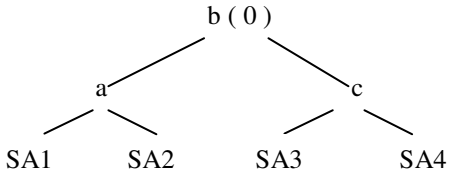
On souhaite insérer 8 dans cet arbre. On obtiendrait :



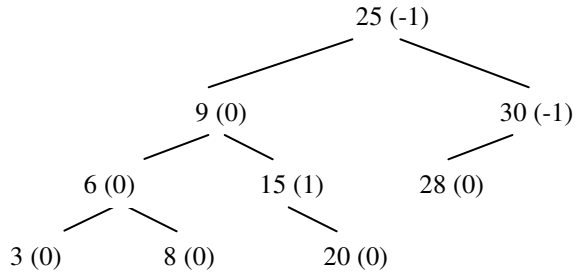
Le facteur d'équilibre du nœud 15 devient égal à -2. Il faut donc réorganiser l'arbre à partir de ce nœud.

D'une manière plus générale, il faut réorganiser l'arbre de la manière suivante. Le sous-arbre du schéma e gauche doit devenir le sous-arbre du schéma de droite.



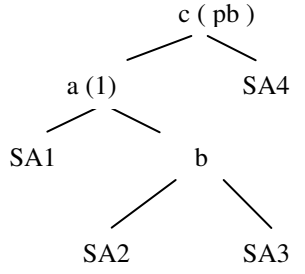
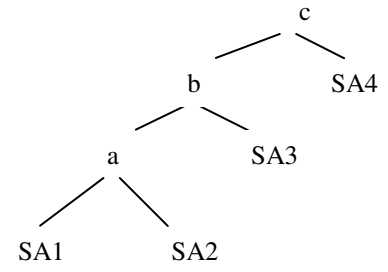
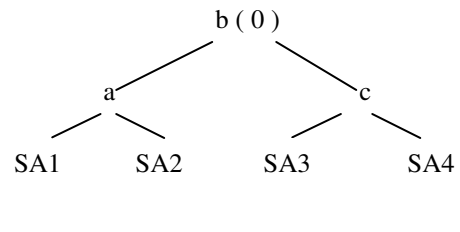


Sur l'exemple, la réorganisation donnera :



L'insertion de 8 provoque un déséquilibre. La réorganisation s'effectue lors de la remontée des appels récursifs (donc dans une séquence d'instructions qui suit l'appel récursif). Le facteur d'équilibre du nœud 15 devrait passer à -2. Il faut donc réorganiser le sous-arbre qui débute par ce nœud. Le nœud qui contient 9 devient racine du sous-arbre à la place de 15 qui glisse à droite du nœud 9. Le nœud 9 est devenu équilibré. Les nœuds en amont de celui-ci ne sont pas concernés par le rééquilibrage.

D'une manière générale cette réorganisation correspond à une double rotation : rotation gauche sur le nœud a, puis rotation droite sur le nœud c.

		
Arbre initial	Après rotation gauche sur a	Après rotation droite sur c

d) Réorganisation de type DG

Le principe est identique à la réorganisation GD précédente, à la symétrie près. Le sous-arbre du schéma de gauche doit devenir le sous-arbre du schéma de droite.

